INF221 — Lecture 1 (2 February 2026)

Hans Ekkehard Plesser, NMBU / REALTEK

Version updated after lecture

Today's topics

1. Introduction
2. Practical information
3. Course overview
4. Insertion Sort—Algorithm and Correctness

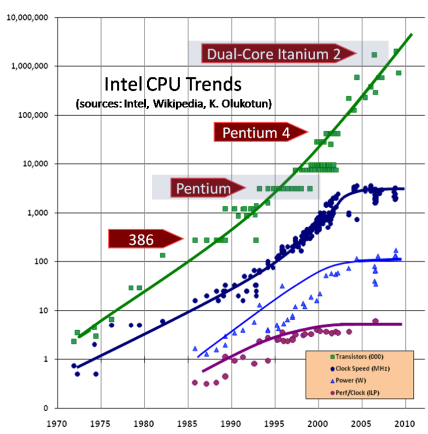
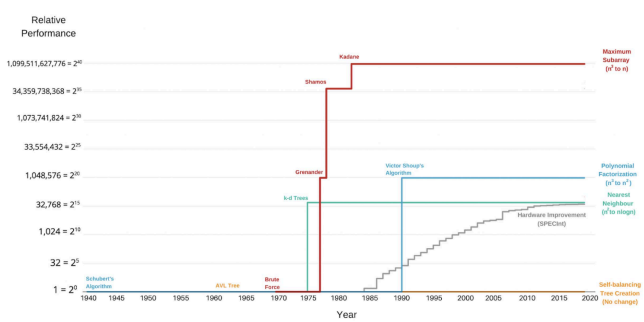
---

1. Introduction

What this course is about

- Proving the **correctness** of algorithms
- Analysing the **performance** of algorithms and data structures
- Key algorithms for e.g., sorting and searching
- Essential data structures such as lists and trees
- Benchmarking: how to measure performance (sensibly)
- Relations between theory and (hardware) reality
- Really hard problems (if there are any)

Is performance to important?

Hardware acceleration	Algorithmic acceleration
	

| The Free Lunch Is Over by Herb Sutter | Sherry & Thompson (2021)

- Solutions are only relevant if they are efficient (time, energy, resources)
- Algorithmic improvements can be *magnitudes* larger than hardware improvements
- Algorithmic improvements are rather cheap (but major improvements are rare)
- Choosing the right algorithms and data structures is essential
- Algorithms and data structures are **essential computing technologies**

Recommended reading

- Sherry, Y., & Thompson, N. C. (2021). *How Fast Do Algorithms Improve? [Point of View]*. Proceedings of the IEEE, 109(11), 1768–1777. https://doi.org/10.1109/JPROC.2021.3107219
  - A nice example of an up-to-date scientific paper that does not need very much in-depth knowledge to read
  - By the end of this course, you should understand terms in the paper that may be confusing now
  - The methods section with equations is at the end of the paper
  - Figures could have been done better
- Nordhaus, W. D. (2007). *Two Centuries of Productivity Growth in Computing*. The Journal of Economic History, 67(1), 128–159. https://doi.org/10.1017/S0022050707000058
  - Looks at hardware acceleration from manual computing to 2007
  - Economists perspective
- The Free Lunch Is Over by Herb Sutter
  - Famous blog post about the need to move to parallel computing
  - Sutter is a key C++ expert and blogger
  - For are recent post, see Software taketh away faster than hardware giveth

    > "our demand for solving ever-larger computing problems consistently outstrips our ability to build greater computing capacity, with no end in sight"

What is an algorithm?

- A **well-defined computational procedure**
- Takes **input**
- Transforms it to **output**
- It thus **solves** a **computational problem**

Example: Sorting problem

- **Input**: A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$
- **Output**: A permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$
- Sample problem **instance**
  - Input: $\langle 31, 59, 41, 26 \rangle$
  - Output: $\langle 26, 31, 41, 59 \rangle$

Typical questions

- How can we solve the sorting problem by computer?
- When can we apply the sorting method?
- Correctness
  - Can we be sure that no data gets lost?
  - Can we be sure that algorithm will always **terminate**?
  - Can we be sure that data is properly sorted?
- Performance
  - How long does it take?
  - What if n doubles, increase 10-, 1000-, million-fold?
  - What if data is, e.g., sorted before we start?
  - How much memory will be required?

---

2. Practical information

Basics

- Teacher
  - Hans Ekkehard Plesser (hans.ekkehard.plesser@nmbu.no)
- Lectures
  - Mondays 08–10
  - Lectures will not be recorded
  - Pre-recorded lecture on 2 March
- Lab
  - Thursdays 10–12 in TF1-205
  - Fridays 12–14 in TF1-105
  - On 5/6 Feb, 12/13 Feb and then every other week (odd-numbered weeks)
  - In-class exercises and work on mandatory assignments
  - Plesser will lead the exercises
  - Sign-up opens in lecture break under `Persons > Lab sessions` in Canvas

Textbook and material

- Textbook
  - Thomas H. Cormen. *Algorithms unlocked*, MIT Press, 2013. ISBN 978-0-262-51880-2.
  - Available at *Boksmia*
  - You are allowed to bring **this book to the exam**
- Compendium
  - Extracts from Cormen, Leiserson, Rivest, Stein (CLRS). *Introduction to Algorithms*, 3rd edition, MIT Press, 2009.
  - Available in the Canvas room for INF221
- Another textbook on the topic
  - Steven Skiena. *The Algorithm Design Manual*, 3rd edition, Springer, 2020
  - Available free of charge while on the NMBU network from Springer Link
  - Website for book: https://www.algorist.com
- Course material
  - Posted in Canvas room for INF221
  - Mostly as Jupyter Notebooks

Prerequisites

- Calculus I (e.g. MATH111/MATH121)
- Programming I (e.g. INF120)
- If you have learned programming in another language than Python or want to refresh your Python, I suggest Jake Vanderplas' Whirlwind Tour of Python.

Mandatory activity

- Three assignments with deadlines on Fridays at 18.00
- Longer assignment after Easter
- Deadlines Fridays at 18.00
  - 20 February
  - 20 March
  - 30 April
- All assignments to be handed in via Canvas
- All assignments to be handed in as Jupyter Notebooks + exported PDF
- You hand in in groups of **three**
  - Register in groups by the end of this week under `Persons > Exercise groups` in Canvas

- Evaluation
  - You get points for each exercise in an assignment
  - Getting a point requires reasonable effort to tackle the exercise
  - 60% of points required across all assignments
  - **No points for assignments delivered late!**
- **Assignments must be approved to pass the course.**
- If you got assignments (mandatory activity) approved in INF221 previously, you do not need to re-do them.
  - **You must inform me by mail by Friday, 6 February, if you want to make use of this rule.**

Final exam

- Written exam on **Wednesday, 27 May 2025** from 09.00 to 12.30
- Grading A–F

---

3. Course overview

1. Introduction (Cormen, ch 1–3; Compendium, pt 1–3)
   A. Introduction
   B. Insertion sort
   C. Analyzing algorithms
   D. Designing algorithms / merge sort
2. Mathematical basics (Cormen, ch 2; Compendium, pt 4–5)
   A. Growth of functions
3. Probabilistic methods (Compendium, pt 6–7)
   A. Randomized algorithms
4. Sorting (Cormen, ch 3–4; Compendium pt 8–10)
   A. Heaps
   B. Heapsort
   C. Priority queues
   D. Quicksort
   E. Medians and order statistics
5. Data structures (Compendium, pt 11–13)
   A. Stacks & queues
   B. Linked lists
   C. Hash tables
6. Trees (Cormen, ch 6; Compendium, pt 14)

   A. Basics of trees
   B. Binary search trees
7. Some algorithms (Cormen, ch 6, 9; Compendium, pt 15–17)
   A. Dynamic programming
   B. Greedy algorithms
   C. Huffman codes
8. Graphs (Cormen, ch 5–6; Compendium, pt 18–20)
   A. Graph basics
   B. Elementary graph algorithms
   C. Shortest-path algorithms
9. Limits of computability (Cormen, ch 10)

Course contents is subject to adjustments!

---

4. Insertion Sort—Algorithm and Correctness

See Compendium, part 1.

The Sorting Problem

- **Input**: A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$
- **Output**: A permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$
- Note: The a_j are sorting **keys**
  - we will usually use numbers in examples
  - could be strings, etc, as long as they are ordered
  - often, we use one feature of a data set as key, e.g., sorting student lists by last name

Keys: an example

First name	Last name	Study program	Birthday	Age	Grade
Joe	Doe	Physics	25/10	23	C
Jane	Dane	Chemistry	08/12	22	B
Alice	Myer	English	02/02	21	A
James	Able	Physics	23/05	22	B

- Each column can be chosen as sorting key

The Insertion Sort Algorithm

Basic idea

1. Take one card/number/name/... that has not been sorted yet
2. Insert in correct place among cards/numbers/names/... that have been sorted already
3. Repeat until no unsorted item left

Extra challenge: Do so **in place**, without requiring extra storage

Pseudocode

```
InsertionSort(A)
for j = 2 to A.length
    key = A[j]
    i = j - 1
    while i > 0 and A[i] > key
        A[i + 1] = A[i]
        i = i - 1
    A[i + 1] = key
```

Notes

- Array indices run from 1 in pseudocode (but from 0 in Python)
- Algorithm manipulates content of array `A`

Python implementation

In [1]:
```python
def insertion_sort(A):
    for j in range(1, len(A)):
        key = A[j]
        i = j - 1
        while i >= 0 and A[i] > key:
            A[i + 1] = A[i]
            i = i - 1
        A[i + 1] = key
```

In [2]:
```python
a = [5, 1, 3, 4, 2]
insertion_sort(a)
print(a)
```

```
[1, 2, 3, 4, 5]
```

In [3]:
```python
b = ['def', 'ab', 'xyz', 'ghi']
insertion_sort(b)
print(b)
```

```
['ab', 'def', 'ghi', 'xyz']
```

Execute in Online Python Tutor

- Run insertion sort in Python Tutor

Proof of correctness

We need to prove that **for any (valid) input**

1. the algorithm terminates
2. the output contains exactly the same keys as the input
3. the output contains the keys in sorted order

What are valid inputs?

- Any array with the following properties
  - The array is *mutable*, i.e., the algorithm can change elements in the array and this change will be visible to the caller of the algorithm when it completes (counterexample: Python `tuple`)
  - Comparison `<` is defined for any pair of array elements
- An empty array is a valid input

Loop invariants

- Key technique for proving statements about algorithms
- Precise statement about the status of data at a certain time during execution
- Prove algorithm correctness using *induction* in three steps:
  - **Initialization:** The invariant is true before the first iteration of the loop
  - **Maintenance:** If the invariant is true before any given iteration of the loop, it is true also before the next iteration
  - **Termination:** When the loop terminates, the invariant provides information about the state of the data which helps us to show that the algorithm is correct (did the right thing)

Loop invariant for Insertion Sort

```
InsertionSort(A)
for j = 2 to A.length
    key = A[j]
    i = j - 1
    while i > 0 and A[i] > key
        A[i + 1] = A[i]
        i = i - 1
    A[i + 1] = key
```

Invariant

At the start of each iteration of the for loop on lines 2–8, the subarray `A[1..j–1]` consists of the elements originally in `A[1..j–1]` in sorted order.

Proof

Initialization

- Before first iteration: `j == 2`
- Therefore `A[1..j–1]` is `A[1..1]`, i.e., `A[1]`
- We have not changed anything in `A` yet, so `A[1]` contains the same elements as it contained originally
- Since `A[1]` contains only a single element, it is in sorted order
- Thus, the invariant holds before the first iteration

Maintenance

- Induction assumption: We assume that the invariant holds for `j == J`, i.e., that `A[1..J–1]` contains the elements originally in `A[1..J–1]` in sorted order
- We now execute the for loop for `j == J`
- We need to show that afterwards, `A[1..J]` contains the elements originally in `A[1..J]` in sorted order
- Proof
  1. On line 3, we store `A[J]` in `key`
  2. On line 4, we initialize `i = j–1 = J–1`
  3. The while-loop on lines 5–7 moves elements to the right
     - It only moves elements one position to the right
     - The rightmost element that is assigned a new value is element `A[J]` (for `i == J–1`)
     - The order of elements is not changed
     - The while-loop stops either
       - if we have copied all elements (`i == 0`); then
         - `A[2..J]` contains the same elements that were in `A[1..J–1]` before we entered the while loop and we know (why?) that these
           - are all larger than `key`
           - are in sorted order
         - we store `key` into `A[i+1]`, i.e., `A[1]` on line 8
       - or if the element at `i` is not larger than `key`; then
         - `A[i+2..J]` contains the same elements that were previously in `A[i+1..J–1]` and we know that these
           - are all larger than `key`
           - are in sorted order
         - `A[1..i]` is unchanged from the previous iteration, i.e., and we know that these
           - are in sorted order
           - the largest element `A[i]` is not larger than `key`

        - we store `key` into `A[i+1]` on line 8
4. Thus
    - All elements in `A[1..i]` are sorted (by assumption) and are not larger than `key`. This holds because we did not touch them in the while loop.
    - All the elements in `A[i+2..J]`
        - are larger than `key`
        - are the same as previously in `A[i+1..J-1]`
        - are sorted (by assumption, because we move every element that we touch in the while loop one position to the right)
    - Since we know that `A[i] ≤ key < A[i+2]` and `A[i+1] == key`, we know that the elements in `A[1..J]` are the same as before and sorted

Termination

- Let `n == A.length` be the number of keys to be sorted
- After the last iteration, we have `j == n+1`
- Thus, the invariant reads: At the start of each iteration of the for loop of lines 2–8, the subarray `A[1..n]` consists of the elements originally in `A[1..n]` in sorted order.
- This is precisely what we want: The array contains the original data in sorted order

In-class questions

1. What happens if `A` is empty or has only one element?
2. What happens if `A` is sorted from before?
3. What happens if we change `A[i] > key` to `A[i] >= key` on line 5?

In [ ]: